

CMPT 383: Assignment #5

Anders Miltner
miltner@cs.sfu.ca

Introduction

This assignment is to help you work with language interoperability. This assignment will likely require the least amount of actual code to be written, but it is quite finicky, and requires a lot of testing to ensure everything works well together. I strongly recommend you dedicate a lot of time to writing your own tests for this assignment.

In this assignment, you will have Rust and Haskell interoperate in writing a program interpreter. While it is a little silly to use interlanguage features for this specific interpreter, this type of usage is important – you can use the high-level language to enable a better high-level API, and the low-level language to build a fast underlying implementation.

In this specific project, the underlying data will be created and transformed all within Rust, with Haskell interpreting the DSL and performing typechecking for the DSL.

I have provided a docker file that you can use for this assignment. This docker file has Rust and Haskell ready on it.

1 Typechecking

First, you should implement the `typecheck` function. In this language we have *overloading* in the `double` function. Because of this, we want to integrate type information into the syntax tree of the program.

Type errors are represented by returning `Nothing`.

The `EInt` AST node has type `TInt`. The `EVec` AST node has type `TVec`, but only when every expression in the expression list has type `TInt` (otherwise it is ill-typed). The `EAdd` AST node has type `TInt`, as long as both provided expressions have type `TInt` (otherwise it is ill-typed). The `EHead` AST node has type `TInt`, as long as the sub-expression has type `TVec` (otherwise it is ill-typed). The `EDouble` AST node should have the same type as the provided sub-expression.

2 Evaluation

Evaluation is tricky. For credit, you must implement `evaluate`, where all computation must be performed in Rust, and all values must be initialized in Rust.

The communication between the two happens in the `Value` type. Values can be either `IntValues` or `VecValues`. Helper functions for transforming between `Value` types and integers or vectors are provided in `helpers.rs`.

To do this, you will need to create different functions to handle the different cases of the `Expr` data type. You should look at the `helpers.rs` and `Helpers.hs` files, as that provides useful examples to build your functions off.

Your `evaluate` should take 3 steps. (1) Typecheck the expression, if the expression does not typecheck, return `Nothing`. (2) Evaluate the expression to an `IO (Ptr Value)`. (3) Show the value using `showValue`.

I recommend following the `showValue` function to help with this process. It is useful for showing how to use one of two different functions depending on the type. Furthermore, the `show_vec` function will be useful for showing how to pull an integer out of a `Vector` (particularly important for implementing `EHead`).